ATLASOPS / PORTFOLIO SERIES

# AtlasOps

## DevOps Portfolio

Build and Operations Handbook

From containers to secure delivery,
observability and recovery.

Docker / NGINX / ModSecurity / Vault

Prometheus / Grafana / Elasticsearch / Logstash / Kibana

CI/CD / Terraform / Ansible / Kubernetes

**DESIGNED FOR A DEVOPS INTERNSHIP PORTFOLIO**

Moroccan hiring evidence + a complete source project

18 chapters / practical runbooks / test evidence / CV guidance

Prepared for Mohammed Abid / 5 November 2026

Local-first learning lab. Runtime verification boundaries are explicit.

# Contents

Use the clickable chapters or PDF bookmarks to move between explanations, operations and evidence.


01 / The project and the evidence 3

02 / Moroccan internship signals 4

03 / Architecture and trust boundaries 6

04 / Prepare the host and get a first success 9

05 / How the implementation was built 11

06 / Docker, Compose and persistent data 13

07 / NGINX, TLS and the WAF 15

08 / Vault, AppRole and hot secret rotation 16

09 / Metrics, dashboards, alerts and reliability 18

10 / ELK and structured logging 20

11 / CI, artifact identity and controlled releases 21

12 / Runbooks and recovery exercises 23

13 / Terraform, Ansible and Kubernetes 26

14 / Verification performed and still required 28

15 / Learning plan, interview demo and CV 30

16 / Architecture decisions 33

17 / Repository map and acceptance checklist 34

18 / Primary technical references 36


## Reading route

Start with Chapters 1-5, run the monitoring profile, then study one subsystem at a time. Chapter 12 is the operational reference. Chapter 14 states what was verified. Chapter 15 helps you turn the work into an honest portfolio story.

# 01 / The project and the evidence

AtlasOps is a complete source repository for a small service with a substantial operational lifecycle. The application is an operations task board. You can create an action, mark it done and delete it; PostgreSQL preserves the records. The DevOps work is everything that makes those simple actions secure, observable, repeatable and recoverable.

The intended learner already knows Docker, Docker Compose and NGINX, has encountered ModSecurity, and wants practical Vault, monitoring and logging experience. New concepts are added where they solve an identifiable problem: Git and CI for safe change; Terraform for infrastructure ownership; Ansible for repeatable host preparation; Kubernetes for application orchestration; health checks, rollback and restore drills for reliability.

**HashiCorp is a company. Vault and Terraform are two of its products.** You are not learning “Vault and HashiCorp” as two equivalent technologies. In this repository, Vault controls access to secrets, and Terraform describes an optional cloud environment.

## What you should be able to demonstrate

A normal request succeeds over HTTPS, while an unauthenticated API request fails. Harmless local XSS and SQL-injection probes are rejected by the WAF. An API key changes without restarting the application. A stopped API produces an actual firing alert. A database dump restores into an isolated database. A new application image passes a health gate or the previous image is restored. Logs connect a request to a response without storing its credentials.

The main platform runs locally and does not require a paid cloud account. Terraform/AWS and Kubernetes are separate, optional exercises. They are included as source and instructions, not represented as deployments already performed in your account.

## Read the validation boundary first

The final test run in this environment passed **25 tests and skipped one PostgreSQL-specific test**. A real Vault 2.1.1 server and Agent were executed. Compose syntax, Prometheus configuration/rules, Alertmanager configuration, Ansible syntax and Terraform formatting/HCL parsing were also checked.

There was **no Docker daemon here**, so the complete container stack, WAF behavior, PostgreSQL integration, ELK ingestion and container release recovery were not executed in this environment. Terraform's provider-backed validation was blocked by a restriction on local Unix sockets. Browser interaction was not exercised. The repository includes the commands and CI jobs to complete these checks. Chapter 14 records the exact boundaries.

Treat the source as a runnable implementation with an explicit acceptance procedure. Do not describe unexecuted cloud, Kubernetes, CI or production outcomes as accomplished work on your CV. Your own completed runs, explanations and incident notes are what make the project credible.

## How to use this book

Start with the market analysis and architecture, then run the smallest useful profile. Read the corresponding chapter while opening the real source files. Change one component, explain your hypothesis, run a targeted check and record the outcome. Operational commands are also maintained in the Markdown runbooks, which are easier to copy than wrapped PDF text.

The book is detailed by design, but you do not need to finish every extension before applying for an internship. A learner who understands a small, tested system can make a stronger case than someone who lists many tools without being able to troubleshoot them.

# 02 / Moroccan internship signals

Research date: **5 November 2026**. This is a small, purposive sample of six distinct internship postings, not a statistical survey of the Moroccan market. Searches used LinkedIn Morocco, Indeed Morocco and ReKrute, with Jobiglo as a cross-check. Cross-posts were deduplicated by employer and role. Required and desirable technologies both count as mentions. A missing mention does not prove an employer does not use a tool.

Some postings are closed, and search-index excerpts may lag the live page. They are evidence of requested skills, not a list of guaranteed open vacancies. Aggregated result totals such as “50+ jobs” were not used as demand estimates.

| ID | Employer / role / location | Observed skill signals | Evidence and status |
|---|---|---|---|
| J1 | CapitaleTech, Cloud & DevOps PFE, Morocco / remote | AWS/GCP, Terraform/Terragrunt, GitHub Actions, Docker, Kubernetes basics, Linux, Python/Bash, secrets, Prometheus/Grafana | Employer LinkedIn page read; shown as 10 months old and closed |
| J2 | Sofrecom Maroc, Automation & DevOps internship, Rabat | Python, Terraform, GitLab CI/CD, Linux, Docker appreciated, REST/JSON/YAML, databases, tests and logs | Employer LinkedIn page read; shown as 3 weeks old and closed |
| J3 | EOL, DevOps internship, Casablanca area | Cloud, Docker, Git, CI/CD, Python/Bash/PowerShell, Terraform/Ansible, monitoring, documentation | Employer LinkedIn page read; shown as 2 months old and closed |
| J4 | Cnexia, Cloud/DevOps PFE, Rabat | GCP, Kubernetes, Docker, Terraform, CI/CD, Linux, scripting, Git, troubleshooting, French/English | Full ReKrute posting read; search listing displayed 10 Sep-10 Nov 2026 |
| J5 | UNACORPORATER, DevOps & Cloud internships, Casablanca | GitHub Actions, Docker, cloud monitoring/backups/security; Terraform and Kubernetes listed as useful extras | Indeed indexed description and listing inspected; direct vacancy page unavailable; opening date/status unconfirmed |
| J6 | MELKO ENERGIE, Cloud/DevOps AWS PFE, Casablanca | AWS fundamentals, GitHub, CI/CD, deployment documentation and a project portfolio | LinkedIn indexed description retrieved; direct page unavailable; status unconfirmed |

## Mentions in this sample

| Skill family | Explicit mentions | Learning decision |
|---|---|---|
| CI/CD | 6 of 6 | Highest priority: pipeline stages, failure gates, artifacts and releases |
| Cloud concepts, including private cloud | 6 of 6 | Learn one provider; AWS is implemented here, GCP is a relevant alternative |
| Docker | 5 of 6 | Go beyond running containers: health, networking, identity and persistent data |
| Terraform / IaC | 5 of 6 | Add a small real infrastructure module and understand state |
| Scripting, explicitly Python/Bash or generic scripting | At least 4 of 6 | Be able to explain and modify the operational scripts |
| Kubernetes | 3 of 6 | Learn Deployments, Services, probes and storage after Compose |
| Linux, explicitly named | At least 3 of 6 | Treat as a foundation even when a listing omits it |

These counts describe only J1-J6 and are not percentages of all Moroccan jobs. CI tooling differs: GitHub Actions and GitLab CI recur; Jenkins is also mentioned in the wider listings. One well-understood pipeline transfers more effectively than three copied pipeline files.

## Early-career cross-check, excluded from internship counts

J7: Sofrecom's **Ingénieur DevOps (F/H) - CDD**, Rabat, published 2 November 2026 on ReKrute, explicitly accepts young graduates and lists Python, GitLab, Docker/Kubernetes/Helm, Terraform/Ansible, Linux, DNS/networking and Vault/Sealed Secrets. This supports the relevance of the additional concepts without turning them all into internship prerequisites.

Vault and ModSecurity were included because they extend your existing interests and create strong security demonstrations. They are not presented as the two most common internship requirements. ELK is useful, but proving CI/CD, Linux troubleshooting and recovery should come first.

Several PFE listings specify Bac+4/5 or final-year enrollment. Describe your 1337/42 education accurately, verify internship-convention eligibility with the recruiter/school, and also search for `stage DevOps`, `stage cloud`, `stage systèmes Linux`, `PFA`, `pré-embauche` and `junior`. Do not label yourself a degree-qualified engineer unless that matches your qualification. A strong portfolio helps demonstrate skill; it does not guarantee eligibility or an offer.

## Source links

- J1: CapitaleTech - LinkedIn Morocco
- J2: Sofrecom internship - LinkedIn Morocco
- J3: EOL - LinkedIn Morocco
- J4: Cnexia PFE - ReKrute · J4 cross-check: Cnexia - Jobiglo cross-check
- J5: UNACORPORATER - Indeed indexed listing (dynamic listing; inspected indexed description)
- J6: MELKO ENERGIE - LinkedIn indexed description (indexed description)
- J7: Sofrecom graduate role - ReKrute

# 03 / Architecture and trust boundaries

## Follow one request

1. The browser resolves `localhost` and connects to host port 8443. Docker forwards that loopback-bound port to the outer NGINX container.
2. NGINX negotiates TLS using the local certificate, limits request size and request rate, assigns a request ID, and forwards the HTTP request to the WAF.
3. ModSecurity evaluates the request using OWASP CRS. Suspicious traffic is blocked before it reaches the API. Accepted traffic continues to `api:8000` on a private Docker network.
4. The FastAPI route reads the currently rendered API key and compares it with the supplied `X-API-Key`. Input validation rejects invalid task fields. SQLAlchemy sends parameterized statements to PostgreSQL.
5. The response returns through both proxies. API metrics record the method, route template, status and latency. JSON log records let you investigate the request later.

The database is the persistent source of task data. Vault is the authoritative store for the API key. The Agent's file is a local cached rendering, not a second secret-management system. Prometheus and Elasticsearch hold operational telemetry; losing them should not delete application tasks.

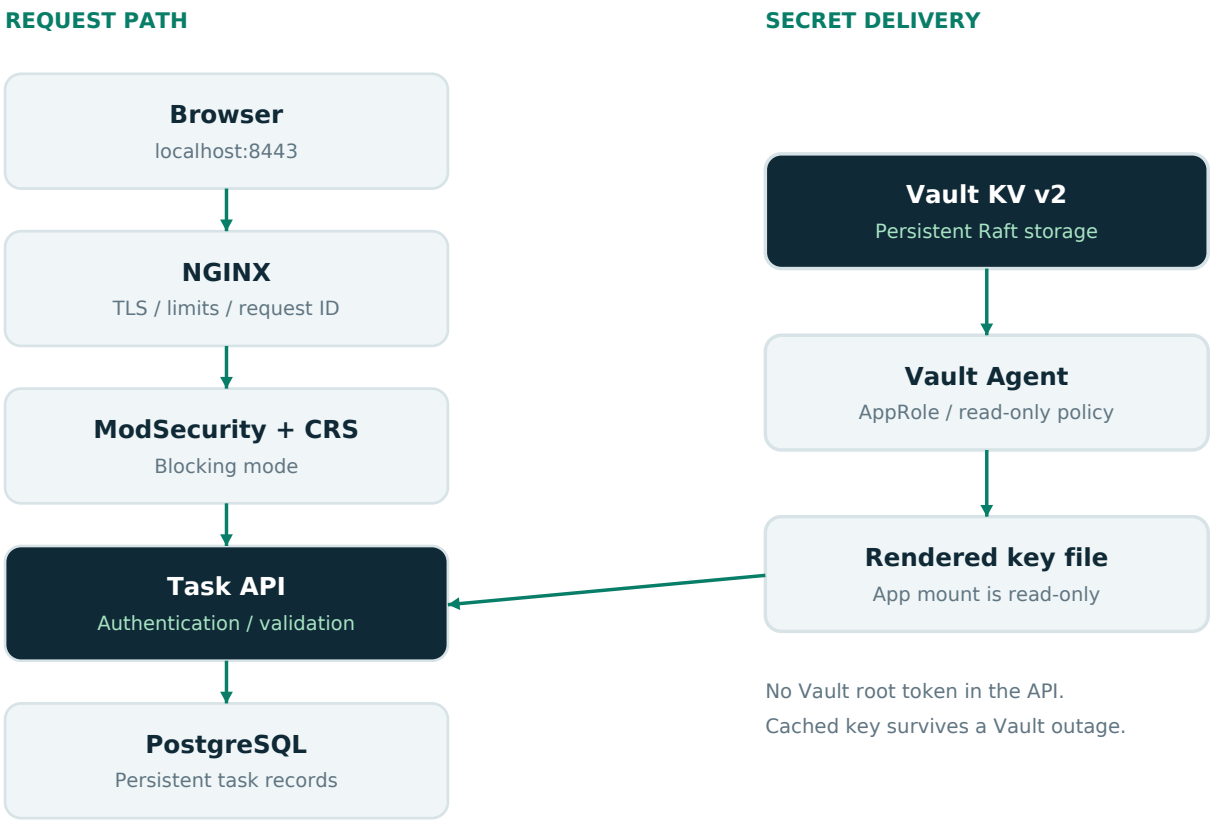
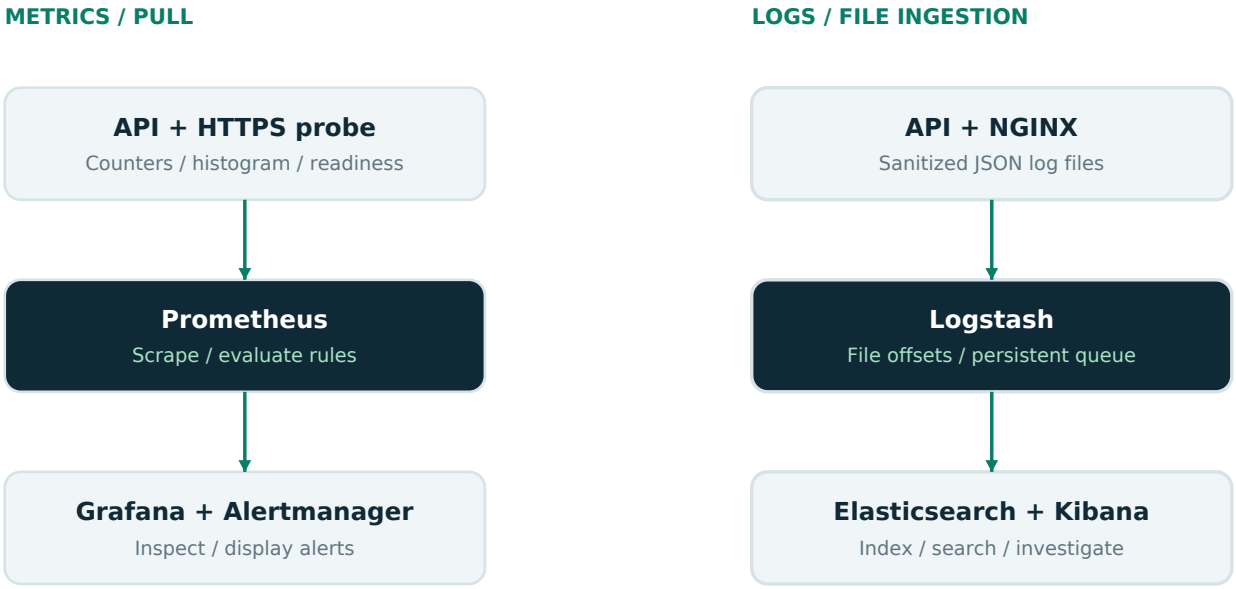



Figure 1. Request processing and the independent secret-delivery path.

## The two independent telemetry paths

Metrics and logs answer different questions. A counter and histogram let you ask “how many requests failed in the last five minutes?” and “how did latency change?” A structured log lets you inspect a particular request ID and status. Prometheus pulls metrics; Logstash tails files and pushes events. Grafana queries the former; Kibana searches the latter.

Blackbox probes `https://nginx:8443/health/ready`. It verifies the certificate against the generated lab certificate and traverses TLS, the WAF and the API's readiness check. The API scrape is a separate path. If the WAF is unavailable but the API still responds internally, API scrape health can stay green while the complete HTTPS probe fails. This distinction is one of the most useful interview explanations in the project.




Two paths, different questions. No Docker socket mounted.

Figure 2. Metrics and logs answer different operational questions.

## Network membership is a form of access control

| Network | Members / purpose | Important consequence |
|---|---|---|
| edge | NGINX and WAF | WAF is reached through the outer proxy; it has no published port |
| application | WAF and API | HTTP application traffic stays inside Docker |
| data | API and PostgreSQL | The database is not reachable through an exposed host port |
| secrets-net | Vault and Agent | The API does not need a Vault network connection or root token |
| telemetry | API, NGINX and monitoring services | Scrapes and probes can reach internal targets |
| logs-net | Elasticsearch, Logstash and Kibana | Log storage is separate from application traffic |

Most networks are marked internal. These networks are not a substitute for a multi-host firewall or encrypted transport: containers sharing a network can still communicate as permitted by that network. Management ports are published on host loopback for the operator. Another process or user with sufficient access to the host may still reach them.

## Ownership boundaries

| Tool | Owns | Does not own here |
|---|---|---|
| Terraform | Optional VPC, subnet, route, security group and EC2 VM | Application containers or application secrets |
| Ansible | Docker repository/packages/service and host prerequisites | Cloud resource lifecycle or business data |
| Compose | Service processes, networks, volume attachments and health dependencies | Off-host disaster recovery |
| Vault | API-key storage and authorization | Automatic rotation of PostgreSQL role passwords |
| GitHub Actions | Tests, scans and publishing a tested image | An automatic SSH deployment into your machine |
| Make | Convenient, repeatable entry points | A scheduler or a replacement for the tools underneath |

Separating these responsibilities prevents two tools from fighting over the same resource. You should be able to name the owner of every important object in the architecture.

## Deliberate limitations

This is a single-host learning platform. It has no multi-node database replication, no public identity system, no guaranteed zero-downtime deployment and no production SLA. ELK authentication is disabled in a loopback-isolated lab. Vault uses local HTTP, one recovery share and retained root recovery material. These choices make the whole lifecycle inspectable; Chapter 12 explains what must change before broader deployment.

# 04 / Prepare the host and get a first success

## Host prerequisites

The documented target is Ubuntu 22.04 or 24.04 on amd64, with Docker Engine and the Docker Compose v2 plugin, Python 3.10 or newer, Make, Git, OpenSSL and curl. Install Docker using its official Ubuntu repository instructions [D1]. The optional Ansible playbook automates that installation for a dedicated VM; it is not a blind workstation reconfiguration script.

Confirm the daemon is accessible before diagnosing the application:

```
docker version
docker compose version
python3 --version
openssl version
make --version
```

Use Compose 2.24 or newer. Docker group membership is effectively root-level access to the host [D2]. If you add your user to that group, reconnect your login session. Do not “fix” Docker socket permission errors with a world-writable socket.

## Choose a profile

| Target | Purpose | Suggested host budget |
|---|---|---|
| make lite | First API, database, WAF and Vault exercises | 4 GB RAM / 2 CPUs |
| make up | Normal portfolio demo with monitoring | 8 GB RAM / 4 CPUs |
| make full | Add all three ELK services | 16 GB RAM / 4 CPUs / 15 GB free disk |

These are conservative planning suggestions, not measured minimums or performance guarantees. Image downloads and Docker itself consume extra disk and memory. If the machine struggles, prove the core behavior before enabling ELK. Running a smaller profile does not stop optional containers already running; use `make down` before reducing the profile.

## First startup

```
cd atlasops
make up
```

`ops.py init` creates `.env` from the example, generates independent random credentials, creates private local directories, and produces a 30-day self-signed certificate. Existing values are preserved. `doctor.sh` checks prerequisites and asks Compose to parse the entire model. The startup script brings up the database and Vault, waits for Vault to be ready, configures AppRole and starts Agent. Finally it starts the selected services and runs smoke/monitoring checks.

The first launch may take several minutes because it pulls images and compiles the application image. The scripts use bounded waits; a permanently broken dependency should produce a visible error rather than hide behind an endless sleep.

## Open the service

Open `https://localhost:8443`. This certificate is self-signed for the local lab, so the browser will warn until you explicitly trust it. The supplied scripts use the exact certificate as a CA file instead of disabling verification. Read the generated client key locally and paste it into the web board:

```
cat .secrets/api-key
```

The interface keeps the key in JavaScript memory for that tab, not localStorage. Reloading clears it. The password input is cleared after connecting. Create a task, mark it done, then refresh the list. Test persistence by stopping and starting the stack normally.

```
make down
make up
```

Normal shutdown preserves named volumes. Do not add `-v` unless you intentionally want to destroy that disposable environment's data.

## Local endpoints

| Component | URL | Credential |
|---|---|---|
| App | https://localhost:8443 | Generated API key |
| Grafana | http://localhost:3000/d/atlasops | admin + .secrets/grafana-password |
| Prometheus | http://localhost:9090 | Host loopback isolation |
| Alertmanager | http://localhost:9093 | Host loopback isolation |
| Vault | http://localhost:8200 | Operator credentials; never the app's root token |
| Kibana | http://localhost:5601 | Host loopback isolation; full profile |
| Elasticsearch | http://localhost:9200 | Host loopback isolation; full profile |

Run `make status` to distinguish running, healthy, unhealthy and exited containers. The one-shot `init` service is expected to finish successfully; it is not a long-running application.

# 05 / How the implementation was built

## Build in layers, prove the contracts

The implementation began with a deliberately small HTTP contract. A service that creates, reads, updates and deletes tasks is enough to make persistence, secrets, metrics, logs and deployment changes observable. The database model has an ID, title, completion flag and creation timestamp. Operations use SQLAlchemy's parameterized statements. The task title is not interpolated into SQL.

The next layer was a repeatable application factory. `create_app()` builds a fresh database engine and metrics registry, registers request handling, and defines startup/shutdown behavior. That makes individual tests independent and avoids Prometheus duplicate-collector failures when more than one app instance exists in a test process.

| Build step | Main files | Contract to verify |
|---|---|---|
| 1. Application | app/main.py, app/static/* | CRUD, validation, authentication, persistence |
| 2. Container | Dockerfile, requirements.txt | Nonroot runtime; same code and pinned dependencies |
| 3. Data and lifecycle | compose.yaml, infra/postgres/init.sh | Persistent storage and least-privilege app role |
| 4. Secret flow | infra/vault/*, scripts/ops.py | Allowed reads, denied writes, rendered key and rotation |
| 5. Ingress | infra/nginx/nginx.conf, WAF image config | TLS, public route restrictions and request filtering |
| 6. Telemetry | infra/prometheus, grafana, logstash | Useful metrics, alerts and searchable structured logs |
| 7. Operations | scripts/*, Makefile | Idempotent startup, backup, isolated restore, release recovery |
| 8. Delivery | .github/workflows/ci.yml | Test/scan gates and the exact tested image |
| 9. Extensions | terraform, ansible, k8s | Clear ownership and explicit optional deployment |

## Application startup

The FastAPI lifespan function creates schema v1 and retries a database connection failure a bounded number of times. This is sufficient for the initial lab schema. It is not a general schema-migration framework. If startup cannot succeed after its retry budget, the process fails so the operator can investigate.

The container runs one Uvicorn worker. Prometheus counters live in that process. Running several workers without a compatible multiprocess metrics strategy would give an incomplete picture depending on which worker answers a scrape. To scale this version, prefer multiple containers/pods with per-instance scrapes, then aggregate in PromQL.

## Public and protected routes

| Route | Behavior |
|---|---|
| GET / | Public operations-board interface |
| GET /health/live | Process responds; does not require database success |
| GET /health/ready | Database can answer and the rendered key is usable |
| GET /metrics | Internal Prometheus exposition; denied at outer ingress |
| GET /api/tasks | Authenticated bounded list, default 50 and maximum 100 |
| POST /api/tasks | Authenticated create; trimmed title of 1-120 characters |
| PATCH /api/tasks/{id} | Authenticated completion-state update |
| DELETE /api/tasks/{id} | Authenticated deletion; missing ID returns 404 |

Malformed UUIDs and invalid payloads return validation errors. Missing or incorrect API keys return 401. A missing/invalid rendered secret or unavailable database produces a sanitized 503. The liveness endpoint can still return 200 during a dependency incident, which is intentional.

## Safe observability in application code

The request middleware starts a timer, calls the handler, identifies the route template and updates the counter and histogram. It uses `/api/tasks/{task_id}` rather than a different label for every UUID. Unmatched routes use a single bounded label. Raw query strings, titles, request bodies, keys and database exception text do not enter the structured application log.

Each log event includes a timestamp, service, request ID, method, route, response status and duration. SQLAlchemy exceptions are converted into a generic database-unavailable response. The UI inserts task titles with `textContent`, so it does not interpret them as HTML. These application controls matter even when a WAF is present.

## A real defect found during construction

The first native Vault integration attempt reached `sys/unseal`, then immediately tried to configure mounts. It failed because the Raft node had not completed leader activation. Bootstrap now polls the health endpoint for active readiness before writing configuration. This is why “the process started” and “the dependency is ready for this operation” are separate contracts.

The later test exercised the real server and Agent successfully. This example is a useful incident story: show the failing assumption, the observed evidence, the narrow correction and the passing verification. Do not present it as an incident in a production company.

# 06 / Docker, Compose and persistent data

## Understand the image

The builder stage downloads wheels from the pinned Python dependency set. The runtime stage installs only those wheels, copies the application and switches to UID/GID 10001. A stable numeric identity makes shared-volume ownership predictable. The Python base image is pinned to a registry manifest digest in addition to its readable tag.

The `.dockerignore` excludes credentials, certificates, backups, state, tests and infrastructure tools from the application build context. This reduces accidental secret inclusion. Docker layers are not a safe place for secrets: deleting a secret in a later layer does not erase it from earlier layers.

The API container runs with a read-only root filesystem, all Linux capabilities dropped and privilege escalation disabled. Its writable paths are a small tmpfs and its log volume. It reads a database secret and a rendered API-key volume. The app's user cannot legitimately modify the read-only mounted API key.

## Understand the Compose lifecycle

`depends_on` with health conditions improves startup ordering [D3], but it is not a continuous dependency manager. If PostgreSQL fails after startup, Compose does not magically migrate the database or fix the application. The application must handle dependency failures, and an operator must investigate. Restart policies and health checks solve different problems: an unhealthy flag is not automatically a restart command.

The one-shot init service establishes ownership on named volumes before nonroot processes need them. Vault and Agent run as UID 10001. Application logs are readable by the log collector, while the rendered secret directory is restricted. This removes the temptation to use a broad `chmod 777` as a generic fix.

## Persistence and the three kinds of files

| Kind | Examples | Lifecycle |
|---|---|---|
| Versioned source configuration | compose.yaml, NGINX config, dashboards, alert rules | Review and commit to Git |
| Generated private host files | .secrets, .certs, .env, .state, .backups | Keep private; do not put into source archives |
| Docker volume data | PostgreSQL, Vault, Prometheus, Grafana, ELK | Survives normal container recreation; requires a backup strategy |

The PostgreSQL initialization script runs only when the data directory is empty. It creates a nonsuperuser application role named `atlas`, revokes public schema creation and grants the permissions required for schema v1. It reads the app password from a file secret.

Changing `.secrets/db-password` after the database exists does not update that role's password. The database and application must agree on the new value during an intentional rotation. This common failure is precisely why password files and database state must be understood separately.

## Compose secrets are not a vault

Compose file secrets are bind mounts granted per service [D4]. On a regular local Compose deployment, they are not automatically encrypted host storage. Several files are readable inside the one container that receives the mount; their host parent directory remains mode 0700. Do not rely on Compose `uid/gid` settings to change ownership for file-backed secrets when the underlying bind mount does not support that mapping.

The API-key source of truth is Vault KV v2. The database and Grafana passwords use local file secrets in this lab. This is an explicit scope boundary; implementing Vault's dynamic PostgreSQL credentials is an advanced follow-up, not a feature implied by the presence of Vault.

## Image identity and upgrades

The example environment pins the principal Docker Hub service images to manifest digests resolved on the research date. `docs/image-manifests.json` records availability checks, which do not prove runtime compatibility or absence of vulnerabilities. Elastic components use a common version. `make lock` can create an override from actually pulled image digests for all infrastructure services.

A tag is a human-readable reference; a digest identifies content. A new application release should use a distinct commit tag or digest. Do not overwrite `demo-v2` and expect a rollback record to preserve the old bytes. Update baseline versions deliberately, regenerate locks where needed, run security scans and rerun the affected acceptance tests.

# 07 / NGINX, TLS and the WAF

## Give each proxy a clear responsibility

The outer NGINX uses a configuration you can inspect directly. It terminates TLS on 8443, redirects HTTP from 8080, sets response security headers, limits request bodies to 16 KB, restricts the request rate and blocks the metrics route. The WAF image supplies a maintained ModSecurity engine plus the OWASP Core Rule Set [D5]. This avoids building a native NGINX module toolchain solely to demonstrate operational controls.

The second hop costs a small amount of complexity, but it separates certificate/ingress concerns from the CRS package lifecycle. The deployment script recreates the WAF after an API image replacement because upstream DNS resolution can retain the old container address until the proxy reloads.

## TLS is transport protection

The generated certificate contains SANs for localhost, nginx and 127.0.0.1. SANs are what hostname validation relies on; a matching common name alone is not a reliable modern substitute. NGINX allows TLS 1.2 and 1.3. The scripts create an SSL context that trusts only the lab certificate for this endpoint.

```
curl --cacert .certs/server.crt   https://localhost:8443/health/ready
```

TLS ends at the outer NGINX. Inside this single-host lab, upstream HTTP and the Vault listener are not TLS-encrypted. Do not describe this as end-to-end encryption. For a distributed deployment, design internal TLS/mTLS, certificate rotation and trust distribution deliberately.

## What CRS contributes

CRS examines patterns associated with common web attacks and computes anomaly scores. The lab enables blocking mode at paranoia level 1. A harmless test request that resembles XSS or SQL injection should receive 403 before reaching the app. `make smoke` also proves a legitimate task creation works, because a WAF that blocks everything is not a successful deployment.

All supplied probes target this local lab. They do not accept arbitrary remote targets. The SQL-looking string test in the Python suite has a different purpose: it calls the application directly to prove the string is treated as data by parameterized SQL, even without the WAF.

A 403 can also be a false positive. Reproduce it, inspect the relevant rule ID in local WAF logs, understand which argument triggered it and create the narrowest justified exception. Do not switch the entire engine to DetectionOnly as a permanent workaround while claiming active protection.

## Rate limiting is another control

The NGINX rate limit is 10 requests per second per source address, with a burst of 30. Requests exceeding the allowed burst receive 429. The sample traffic generator is intentionally small and bounded, approximately five requests per second. This is a demonstration, not a load-capacity benchmark.

The UI key is not a user identity. A shared key is enough to demonstrate secret delivery and rotation; it does not provide per-person roles, account recovery or detailed authorization. Add a proper identity system only after defining those product requirements.

## Do not leak the protected path

The API, WAF and database publish no host ports. The outer ingress rejects `/metrics` and its subpaths. NGINX access logs contain a request ID and path but omit query strings, bodies and credential headers. WAF audit settings omit request headers and bodies. Never put credentials in query parameters, and inspect screenshots before sharing them.

# 08 / Vault, AppRole and hot secret rotation

## The lifecycle that development mode hides

A normal Vault server starts with encrypted persistent storage. On first use it must be initialized; initialization produces recovery/unseal material and an initial root token. The lab stores that material in a private host file so the workflow is repeatable. After a server restart, the node seals and must be unsealed before it can serve secret operations [D6].

The lab uses integrated Raft storage with one node, one share and a threshold of one. This teaches the lifecycle but provides no node redundancy and concentrates recovery authority. `disable_mlock` is an explicit container-lab setting. Keep the host and its swap/security posture in mind; this is not a hardened production Vault deployment.

## Bootstrap once, re-enter safely

`vault-bootstrap` waits for the listener, checks initialization state, initializes only an empty Vault, unseals it and then waits for an active healthy leader. If persistent data exists without matching recovery material, it stops. If recovery material exists but the Vault volume is unexpectedly empty, it stops rather than overwriting your old recovery information.

Once ready, it enables a KV v2 mount and AppRole authentication, installs a policy, writes or preserves the API secret, creates an AppRole and issues a 24-hour SecretID. On later starts it preserves the existing key and refreshes the bootstrap credential. Vault audit logging is enabled separately from the application log pipeline.

## The least-privilege policy

```
path "secret/data/atlasops" {
  capabilities = ["read"]
}
```

KV v2's data API path includes `/data/`; its metadata path is a different API path. A policy for the wrong path can authenticate successfully but still deny the secret read. The real integration test verifies that this role reads its assigned secret and cannot write it or read an unrelated path.

The application never receives the root token. Agent receives an AppRole RoleID and SecretID. RoleID identifies the role; SecretID is a bootstrap credential. Protect both, especially SecretID. Agent authenticates and manages its token renewal/re-authentication lifecycle [D7].

## File rendering separates the application from Vault

Agent reads `secret/data/atlasops` and renders the `api_key` field into `/run/atlasops/api-key`, with restrictive permissions. The API sees that volume read-only and opens the file for every authenticated request. It therefore needs neither a Vault client library nor a network connection to Vault.

Static KV v2 secrets do not behave like leased dynamic database credentials. The Agent template polls them; the lab interval is 10 seconds [D8]. Agent renders a replacement file when the value changes. The API picks up that new file on its next authentication check.

## Demonstrate rotation

```
make rotate-key
```

The script generates a new random value, writes it to KV v2 and saves the new local client key. It waits until the new key is accepted, then checks the old key receives 401. It does not restart the API. If the Agent is down, the new value remains stored but the app can continue using its previously rendered value until the Agent catches up.

This behavior is an availability tradeoff, not instantaneous revocation. Sealing Vault or revoking an Agent token does not magically erase a secret already rendered to a file. If your application needs immediate revocation, design an online authorization check or a short-lived credential scheme with clear expiry semantics.

## Expiry and recovery

The AppRole token has a 10-minute TTL and one-hour maximum renewal lifetime. Agent can re-authenticate while the 24-hour SecretID remains valid. After that, issue a new one with `make vault-refresh`. Restarting the Agent in that command ensures it uses the refreshed credential files.

`make vault-snapshot` exports an integrated-storage snapshot. A snapshot, its matching recovery material and an actual isolated restore exercise belong in a disaster-recovery plan. This repository creates snapshots but does not claim a completed Vault snapshot-restore drill.

## Next step after you understand this version

Replace the lab root workflow with a constrained operator identity. Use multiple recovery holders or cloud auto-unseal, TLS for the Vault listener, short-lived workload identity, off-host encrypted snapshots and a documented audit-log retention policy. On Kubernetes, learn Kubernetes authentication rather than copying a permanent AppRole SecretID into every pod.

# 09 / Metrics, dashboards, alerts and reliability

## Instrument a question, not a screenshot

The API exposes a request counter and latency histogram. The useful questions are request volume, error ratio and response time. They are commonly called RED metrics: rate, errors and duration. The six-panel dashboard adds internal scrape health and complete HTTPS readiness. It is provisioned from source, so restarting or recreating Grafana does not require manual dashboard reconstruction [D9].

Prometheus discovers fixed service names through Compose DNS and scrapes every 15 seconds. Alert expressions are also evaluated every 15 seconds. These intervals affect detection delay; no one-second alerting claim is made.

## Three PromQL ideas to explain

```
sum by (route) (
  rate(atlasops_http_requests_total{route=~"/api/.*"}[5m])
)
```

A counter only increases until its process restarts. `rate` estimates changes per second over a range and accounts for resets. `sum by (route)` combines instances while preserving the route distinction. The API filter prevents routine health probes from hiding application traffic behavior.

```
histogram_quantile(0.95,
  sum by (le) (
    rate(atlasops_http_request_duration_seconds_bucket[5m])
  )
)
```

This estimates a 95th percentile from histogram buckets. It does not mean every request is below that value, and it can be noisy or absent with very little traffic. The actual dashboard filters API routes. Do not average already-calculated per-instance p95 values and assume you obtained a global p95.

Error ratio divides 5xx request rate by total request rate. The dashboard supplies a zero numerator when no error series exists, and clamps the denominator to avoid division by zero. Its threshold is a teaching target, not a measured service-level agreement.

## Labels can break a monitoring system

Every unique combination of labels creates a time series. Adding task IDs, user IDs, arbitrary URLs or API keys can create unbounded cardinality and leak sensitive information. The application uses route templates and a bounded method set. The tests verify that UUID values do not appear as metric labels.

## Alerts need persistence and routing

| Alert | Trigger | Pending period |
|---|---|---|
| AtlasApiDown | API scrape target is down | 1 minute |
| AtlasHttpsUnavailable | Complete HTTPS readiness probe fails | 1 minute |
| AtlasHighErrorRate | More than 5 percent API 5xx responses | 2 minutes |
| AtlasHighLatency | Estimated p95 above 500 ms | 3 minutes |
| AtlasCertificateExpiring | TLS certificate expires within 7 days | 5 minutes |

The `for` duration requires the condition to stay true before the alert fires [D10]. Alertmanager groups and routes alerts to a local demonstration receiver. The UI displays them, but no email or chat messages are sent. Integrating an authorized external destination is a later configuration task, not a capability silently enabled here.

The promtool rule test verifies that the API-down alert is not firing too early and is firing after its pending interval. `make drill` verifies the runtime condition against the real Prometheus HTTP API when you run the stack on Docker.

## SLI, SLO, RPO and RTO

An SLI is a measured indicator, such as the fraction of successful requests. An SLO is a target for that indicator over a stated window. “99.5 percent success over 30 days” is a proposed target until you collect enough valid data. A short laptop demo cannot prove a month-long availability result.

RPO is acceptable data loss; RTO is acceptable recovery duration. A backup schedule influences RPO, and a tested restore process influences RTO. Use these terms in the interview only with a concrete example and an honest statement of what you measured.

# 10 / ELK and structured logging

## Why the application writes JSON

The API writes one JSON event per request: timestamp, service, request ID, method, route, status and duration. NGINX uses the same broad shape. Different duration-field names preserve units: milliseconds for API logs, seconds for NGINX logs.

The API also writes to stdout for `docker compose logs`. Its file handler rotates at 2 MB with three backups. Docker's JSON log driver is limited separately: configuring one storage path does not limit the other.

## Logstash's responsibilities

The file input reads active and rotated JSON logs from a read-only shared volume. Its codec parses the JSON. A date filter turns the source timestamp into Elasticsearch's `@timestamp`. A mutation removes file/host bookkeeping fields that are not needed for the demonstration. The output writes daily `atlasops-YYYY.MM.dd` indices.

The sincedb file remembers how far Logstash has read. It lives in a named data volume rather than `/dev/null`, so a restart does not deliberately replay the entire log history. A persistent queue buffers events awaiting delivery [D11]. Neither mechanism guarantees exactly-once delivery or unlimited buffering. A crash or rotation can still cause duplicates, and a full disk remains an operational incident.

## Elasticsearch and Kibana setup

All three Elastic components use the same version. Elasticsearch runs as one node with a small heap. `node.store.allow_mmap=false` avoids host kernel tuning for this lab; production tuning has different requirements [D12].

`make elastic-setup` installs a seven-day lifecycle policy and an index template, applies retention/replica settings to already created lab indices, and creates the Kibana data view. The one-node template uses zero replicas because no second node exists to host them. Zero replicas is not a resilience feature.

The management ports bind to loopback and the stack uses an internal network. Elasticsearch/Kibana authentication is disabled only for this local exercise. A public deployment requires a different security configuration; the WAF in front of the application does not protect a separately exposed Elasticsearch endpoint.

## An investigation exercise

Start full mode, create a task, make an invalid authenticated request, and run the supplied WAF probes. In Kibana, select a recent time range. Search by `service`, response `status` or a request ID. Ask which layer returned the error. A WAF block is visible as an ingress 403 without a matching successful application operation.

The shared file pipeline collects API and outer NGINX logs. It does not claim to ingest every service's stdout or the full WAF audit stream. Check the WAF's own local logs when you need a rule ID. This deliberate boundary reduces sensitive transaction data in the search index.

## Rotation and retention are separate

`make rotate-logs` renames the NGINX file and asks NGINX to reopen its log destination. Logstash can continue following file offsets. Elasticsearch's seven-day retention deletes old indexed data, not the original NGINX file. A host operator still needs to invoke rotation regularly for a long-running installation; no background schedule is created by this project.

Vault audit logs need a separate retention/access plan. PostgreSQL backups are also separate: ELK does not provide database recovery or a complete security audit archive.

# 11 / CI, artifact identity and controlled releases

## What happens on a pull request

The workflow checks out source, installs the pinned dependencies and runs the tests. A PostgreSQL service enables the database integration test that was skipped in the generation environment. Trivy scans source dependencies and likely embedded secrets. Terraform formatting and provider-backed validation run on the CI host.

After these checks pass, the stack job builds and starts the Compose platform with monitoring. It verifies API/TLS/WAF behavior, native Prometheus and NGINX configuration, key rotation, a real backup/isolated restore, and an actual alert/recovery drill. An optional manual workflow input enables full ELK validation. The built API image is then scanned.

A security gate may fail because an advisory changes after this repository was generated. Inspect and fix the finding, upgrade relevant dependencies and rerun the affected checks. Do not silently turn `exit-code 1` into success while claiming a clean image.

## Publish what was actually tested

The stack job exports the tested API image as a compressed workflow artifact. The publication job loads that artifact and tags it with the commit SHA. It does not rebuild a potentially different image after testing. Publication only runs for successful pushes to `main`; pull requests cannot publish.

GitHub's built-in token receives `packages: write` only in the publishing job, and credentials are passed to `docker login` through standard input [D13]. You must create your own repository and enable the appropriate package permissions. No repository or public package has been created on your behalf.

## CI versus delivery versus deployment

Continuous integration checks that a change is compatible with the project. Publishing a versioned image makes it available for delivery. Deploying it changes a running environment. Here deployment is an explicit host-side `make deploy` command. This is intentionally a controlled release workflow, not a claim of autonomous production deployment.

The host operator can inspect the CI evidence and choose a specific commit tag or digest. If the package is private, authenticate Docker on that host with an appropriately scoped credential. Do not paste a long-lived token into the Makefile or commit it to `.env.example`.

## The release algorithm

1. Inspect the currently running API image and version. Refuse to proceed if the stack is not running or the new image reference is not distinct.
2. Use an existing local release image or pull it from the registry.
3. Start the new API image without rebuilding; wait for readiness within a bounded interval.
4. Recreate the WAF and outer NGINX so the request path points at the new upstream.
5. Run the complete smoke test. Only on success record the new desired image in `.env` and preserve the old image/version in `.state`.
6. If any gate fails, attempt the same switch procedure back to the previous image and report whether recovery succeeded.

This process can interrupt requests. It is a single-host release exercise, not blue/green or a zero-downtime guarantee. The smoke test temporarily creates and deletes one task. Keep operational test data

distinguishable from real business records if you extend the application.

## Database compatibility limits rollback

The current version creates schema v1 without destructive migration steps. If you add incompatible schema changes later, rolling back the image may not restore service. Learn expand/migrate/contract changes: add compatible schema first, migrate data, switch code, and only later remove the old shape. A rollback plan must include database compatibility, not just a saved Docker tag.

## Supply-chain improvements after the basics

Pin third-party workflow actions to reviewed commit SHAs, generate an SBOM, add image provenance/signing, restrict registry permissions, and validate signatures in a deployment policy. These are follow-up exercises. The current workflow supplies vulnerability/secret gates and versioned publication; it does not claim signing or verified provenance.

# 12 / Runbooks and recovery exercises

## Start, stop and inspect

```
make up
make status
bash scripts/compose.sh logs --tail=80 api vault-agent nginx
make down
```

Use `make full` for ELK. `down` preserves data. There is intentionally no convenient production-data wipe target. Never use `docker compose down -v` on a lab you want to preserve. CI uses `-v` only on its disposable runner.

## API unavailable

1. Check the external path: `curl --cacert .certs/server.crt https://localhost:8443/health/ready`.
2. Check `make status`. A running container can still be unhealthy.
3. If the API cannot connect to PostgreSQL, inspect `bash scripts/compose.sh logs --tail=60 db api`. Check service DNS, initialized database roles and password-file consistency.
4. If the rendered secret is missing, inspect Vault and Agent health. Run `make vault-unseal` after Vault restarts; run `make vault-refresh` when the AppRole SecretID expires.
5. If the API is healthy but ingress fails, inspect NGINX and WAF logs; validate configurations with `nginx -t` inside both containers.
6. If an IP changed during a manual API recreation, recreate the WAF so it resolves the new upstream. The release script already does this.
7. Confirm recovery with `make smoke` and wait for the next Prometheus evaluation to resolve alerts.

`/health/live` only proves the process responds. `/health/ready` also checks the database and a usable rendered API key. Do not restart an otherwise healthy application repeatedly just because a dependency is temporarily down.

## Demonstrate an incident

Run `make drill` with monitoring active. It stops the API, waits for the actual `AtlasApiDown` firing state and starts the API again. Its exit trap attempts recovery even when interrupted. Watch Prometheus Alerts and Alertmanager locally. Capture the observed detection and recovery times in `docs/EVIDENCE_TEMPLATE.md`; do not replace them with guessed results.

For a database-only incident, use `bash scripts/compose.sh stop db`, observe readiness failures while process liveness remains up, then `bash scripts/compose.sh start db`. Run this manually on your disposable lab, not during a release. API reconnection uses SQLAlchemy pool pre-ping. If startup happens while DB is unavailable, bounded startup retries eventually fail visibly.

## Rotate the API key

```
make rotate-key
cat .secrets/api-key
```

The script writes a new value to Vault KV v2 and saves the new local client key. Agent polls static secrets every 10 seconds and atomically replaces its rendered file. The API re-reads that file per authenticated request. The script verifies the new key works and the old key fails; it does not restart the API. Allow up to 90 seconds for the bounded check.

If confirmation times out, do not lose the new key or keep retrying rotations. Inspect Agent health, use `make vault-refresh`, and test the current `.secrets/api-key`. The browser holds the old key in memory until you reconnect it.

The AppRole token lasts 10 minutes and can renew up to one hour; Agent re-authenticates as needed. The bootstrap SecretID lasts 24 hours. Reissue it with `make vault-refresh`. This bounded credential is deliberately not an unattended production identity solution.

## Vault restart and recovery

Vault runs in persistent server mode, not `-dev`. Its Raft data survives normal restarts. After a process restart it seals; `make vault-unseal` uses the local recovery share. If `.secrets/vault-init.json` is missing for an existing Vault volume, startup stops and requires the original material. It never silently reinitializes or overwrites a mismatched Vault.

`make vault-snapshot` writes a Raft snapshot into `.backups`. Treat it and recovery material as sensitive. The repository does not implement automated Vault snapshot restore; follow the official Raft restore guide in an isolated clone and preserve the original unseal material. A Vault snapshot alone is not a PostgreSQL backup.

Production upgrades: multiple independently held recovery shares or cloud auto-unseal; TLS between clients and Vault; audited human access; short-lived workload identity; root-token revocation after configuring a replacement administrator; off-host encrypted snapshots. Do not revoke the lab root token first and then expect its helper scripts to keep working.

## Database backup and isolated restore

```
make backup
# Replace the following path with the printed output:
make restore-test FILE=.backups/postgres-YYYYMMDDTHHMMSS-NNN.dump
```

`pg_dump --format=custom` produces a logical snapshot. The script writes a temporary file, renames only on success and records SHA-256. The restore script checks the hash, creates a unique temporary database in the same PostgreSQL instance, restores in a single transaction and queries `tasks`. An exit trap removes only that temporary database. It never replaces `atlasops`.

A passing restore drill confirms that this dump can be restored and queried, not that you have automatic disaster recovery. An off-host copy is still required to survive loss of the VM. Define an RPO (maximum acceptable data loss) and RTO (maximum acceptable recovery duration), choose a backup frequency, then measure the restore. A proposed daily backup implies up to about 24 hours of data loss; this project does not schedule that job for you.

## Release and rollback

```
docker build -t atlasops-api:demo-v2 .
make deploy IMAGE=atlasops-api:demo-v2 VERSION=demo-v2
make rollback
```

Use a unique tag or registry digest for every release. The script inspects the running image, checks the new one, switches the API, recreates proxies, runs the complete smoke test and only then records the new desired image in `.env`. If a gate fails, it attempts to restore the previous image and reports failure. The previous release is in `.state/previous-release.json`.

This process can interrupt traffic. Database schema v1 is created at app startup; no destructive migrations run. If you later add schema changes, use an expand/migrate/contract approach and test old/new compatibility. Rolling back an image does not undo an incompatible database migration.

## Investigate logs

After `make full`, open Kibana Discover, select the `atlasops-*` data view, and set the time window to the last 15 minutes. Try:

```
service: "atlasops-api" and status >= 500
service: "nginx" and status: 403
request_id: "COPY_A_REAL_REQUEST_ID"
```

API logs use route templates, status, duration and request IDs. They omit bodies, keys and query strings. NGINX logs also omit query strings and headers. WAF audit output excludes request headers/bodies; it is not part of this file-ingestion pipeline. An ingress 403 plus an isolated WAF log check is the expected investigation route.

Logstash remembers offsets in its data volume and has a persistent queue. Delivery is not exactly-once: a crash or rotation can produce duplicates. The API rotates four 2 MB files. Run `make rotate-logs` to rotate NGINX logs; automation of that task is left to the host operator. Elasticsearch indices use seven-day retention after `make elastic-setup`. Vault audit logs are separate and also require an operator retention policy for longer use.

If Discover is empty: run `make load`; check `make verify-full`; inspect Logstash for parse or permission errors; check Elasticsearch health and the selected time window. Restarting Logstash should not require deleting its sincedb.

## Common failures

| Symptom | Likely cause / action |
|---|---|
| Permission denied for Docker | Start the daemon, use appropriate permissions, reconnect after group membership change |
| Port already allocated | Stop the conflicting local service; do not casually expose the lab on all interfaces |
| Certificate error | Expired 30-day certificate or wrong hostname/CA; renew in the local lab and restart NGINX/Blackbox |
| 401 after rotation | Reconnect the web board with the new key |
| Vault sealed after restart | `make vault-unseal` |
| Agent auth errors after a day | `make vault-refresh` |
| Database password authentication failed | Existing DB retained its original password; restore matching secret or rotate role deliberately |
| 502 after manual API replacement | Recreate WAF and NGINX; use release script next time |
| ELK exits / is slow | Check memory limits and host resources; use monitoring profile first |
| Grafana says No data | Generate traffic and wait for scrape/rate windows; inspect Prometheus targets |
| Trivy fails CI | Inspect the exact advisory and upgrade/retest; do not convert the gate to an unconditional pass |

## Certificate renewal in this disposable lab

Keep a copy of the old cert if needed, then generate a replacement with the same SANs. This operation changes local trust; update the dedicated browser profile and restart the probe to reload its CA. Never replace certs belonging to another service.

```
openssl req -x509 -newkey rsa:2048 -sha256 -nodes -days 30 \
  -keyout .certs/server.key -out .certs/server.crt \
  -subj '/C=MA/O=AtlasOps Lab/CN=localhost' \
  -addext 'subjectAltName=DNS:localhost,DNS:nginx,IP:127.0.0.1'
chmod 600 .certs/server.key
chmod 644 .certs/server.crt
bash scripts/compose.sh restart nginx blackbox
make smoke
make verify
```

# 13 / Terraform, Ansible and Kubernetes

These extensions are source-complete learning exercises. They were not applied to your accounts or cluster. The Compose project is the main reference implementation.

## Terraform: a dedicated AWS VM

Terraform owns cloud resources; Ansible owns OS package configuration; Compose owns containers. Do not also manage those same containers with a Terraform Docker provider.

Prerequisites: an AWS account, authorized short-lived credentials, Terraform 1.6+, an SSH key, and an explicit budget. Set an AWS billing alert in your account before provisioning. EC2, EBS, public IPv4 and traffic can incur charges; no free-tier promise is made. An apply requires your own deliberate action. The default `t3.large` is intended for core/monitoring; consider a 16 GB instance for ELK.

```
cd terraform/aws
cp terraform.tfvars.example terraform.tfvars
# Edit: admin_cidr must be your public IPv4/32; provide your PUBLIC SSH key.
terraform init
terraform fmt -check
terraform validate
terraform plan -out=lab.tfplan
# Review every resource and expected cost before your optional apply:
terraform apply lab.tfplan
terraform output -raw public_ip
```

The module creates an isolated VPC/subnet, a default route through an Internet Gateway, a security group allowing SSH from one `/32`, a key pair and a VM with encrypted root storage and IMDSv2 required. The Ubuntu image is selected from Canonical's owner account. The selected AMI ID is output; pin that ID in a future immutable environment instead of always selecting the most recent image.

Only port 22 is reachable from your allowed source. No application, Vault, database or dashboard port is opened publicly. The VM uses outbound internet access to install packages and pull images. There is no NAT Gateway, managed database or load balancer in this small module.

Terraform state is local for this exercise and is ignored by Git. Keep it securely; it may contain sensitive infrastructure metadata. Teams should use a secured remote backend with encryption, locking and controlled access. Do not put API keys or Vault secrets into Terraform variables and assume `sensitive = true` keeps them out of state.

## Ansible: prepare the host

Install Ansible in a local virtual environment (`pip install ansible-core`). Copy `ansible/inventory.example.ini` to `ansible/inventory.ini`, replace host/user/key values, and verify SSH host identity independently. Host-key checking is not disabled.

```
ansible-playbook -i ansible/inventory.ini ansible/playbook.yml --syntax-check
ansible-playbook -i ansible/inventory.ini ansible/playbook.yml
```

The playbook targets a dedicated Ubuntu 22.04/24.04 amd64 host. It installs base utilities, configures Docker's signed apt repository, installs Engine/Compose, enables Docker and adds the SSH user to the Docker group. This group is root-equivalent. Reconnect SSH after it changes. A second run should largely report `ok`; inspect any changes rather than promising zero changes while package metadata evolves.

Copy only clean source to the VM; generate fresh secrets there. For example, create a clean source archive using Git, transfer it with `scp`, extract into `~/atlasops`, then run `make up`. Do not copy your local `.secrets`, backups, `.env` or Terraform state into a public repository.

```
# Local repository; requires an initial Git commit:
git archive --format=tar.gz --output=/tmp/atlasops-source.tar.gz HEAD
scp /tmp/atlasops-source.tar.gz ubuntu@VM_IP:~/
# On the VM:
```

```
mkdir -p ~/atlasops
tar -xzf ~/atlasops-source.tar.gz -C ~/atlasops
cd ~/atlasops
make up
```

Use tunnels from your laptop. Stop your local Compose stack first if those local ports are occupied:

```
ssh -N -L 8443:127.0.0.1:8443 -L 3000:127.0.0.1:3000 \
  -L 9090:127.0.0.1:9090 ubuntu@VM_IP
```

The browser then uses the VM's certificate through `https://localhost:8443`. For curl from your laptop, copy only the VM's public certificate and trust that file explicitly. Retrieve the VM's generated client key through your private SSH session. This is a private demo VM, not a public website deployment.

When finished, take any required off-host backups and review `terraform plan -destroy`. `terraform destroy` permanently removes the lab VM and its root disk/data. Run it only for this dedicated lab after deciding what to preserve. No destroy action has been performed by this project generator.

## Kubernetes: migrate the application to kind

Install Docker, kind and kubectl from their official sites. Run:

```
make k8s-up
kubectl --context kind-atlasops -n atlasops port-forward \
  --address 127.0.0.1 svc/api 8088:8000
```

Open `http://localhost:8088` and use `.secrets/api-key`. All commands target the named `kind-atlasops` context. This separate cluster has its own PostgreSQL data volume; your Compose tasks will not appear there.

The exercise contains a PostgreSQL StatefulSet with a PVC; a ConfigMap for database initialization; scoped credential volumes; a Deployment; a ClusterIP Service; startup, readiness and liveness probes; nonroot/read-only API settings; resource requests/limits; and a service account token disabled for pods that do not need Kubernetes API access.

After the first successful startup creates schema v1, practice scaling and recovery:

```
kubectl --context kind-atlasops -n atlasops scale deployment/api --replicas=2
kubectl --context kind-atlasops -n atlasops get pods
kubectl --context kind-atlasops -n atlasops rollout restart deployment/api
kubectl --context kind-atlasops -n atlasops rollout status deployment/api
```

To learn an image rollout, build a **different tag**, load it with `kind load docker-image IMAGE --name atlasops`, use `kubectl set image deployment/api api=IMAGE`, inspect rollout status, then `kubectl rollout undo deployment/api`. Reloading the same local tag alone does not prove an image change.

The Kubernetes exercise intentionally includes only the app/database migration. It uses a snapshot of the local key in a Kubernetes Secret; it has no live Vault integration, WAF, TLS ingress, metrics stack, log stack, NetworkPolicy enforcement, HPA or multi-node HA. Kubernetes Secrets are not inherently encrypted simply because their YAML representation is base64. Next tasks are Vault Kubernetes auth, an ingress controller, Helm packaging, a metrics collector and appropriate network controls.

`make k8s-down` deletes only the named kind cluster, including its lab database. Preserve anything you care about first.

# 14 / Verification performed and still required

Checked on **5 November 2026**. This record distinguishes executable tests, configuration checks and work requiring a Docker/cloud host.

| Check | Result | Meaning / limitation |
|---|---|---|
| Python test suite with native Vault enabled | **25 passed, 1 skipped**, 27.83 seconds | Includes application behavior, persistence with SQLite, configuration checks and a real Vault/Agent integration test |
| PostgreSQL-specific test | **Skipped here** | No PostgreSQL server was available; a real PostgreSQL service is configured in CI |
| Real Vault 2.1.1 + Vault Agent | **Passed** | Persistent Raft server initialization, leader readiness, AppRole login, allowed read, denied write/unrelated read, template rendering, key rotation, snapshot, seal/unseal |
| Docker Compose 2.39.4 configuration | **Passed** | Full monitoring/logging model parsed by Compose; no containers executed |
| Prometheus 3.13.5 promtool | **Passed** | Five alert rules, alert timing test and config syntax; the native config check changed only the mounted rule-file path to a local path |
| Alertmanager 0.28.1 amtool | **Passed** | Configuration valid; one local receiver, no external delivery |
| Ansible 2.21.6 syntax check | **Passed** | No host provisioning was performed |
| Terraform 1.13.4 formatting and HCL parsing | **Passed** | AWS provider 6.72.0 initialized and lock file created |
| Terraform provider-backed validate | **Blocked by this runtime** | Provider could not create its local Unix socket: operation not permitted. Run terraform validate and plan on your machine; neither is claimed passed |
| JavaScript syntax check | **Passed** | Browser automation not performed; Chromium download was unavailable |
| Docker Hub / Elastic image manifests | **14 references resolved** | Availability only; see image-manifests.json. This is not execution or a security scan |
| Full Docker stack, WAF runtime, PostgreSQL backup/restore, ELK ingestion and release recovery | **Not executed here** | Docker daemon unavailable. Executable smoke, verification, configuration and drill commands are supplied; CI runs these on a Docker runner |
| AWS apply / Ansible run / kind cluster | **Not executed** | No account or host mutations, no cloud spend |

| Check | Result | Meaning / limitation |
|---|---|---|
| Trivy security scans / GitHub Actions workflow | **Not executed here** | Workflow is supplied; a green CI run and clean scans are not claimed |

The test suite emits one upstream Starlette warning about future migration from httpx to httpx2 in its TestClient. Tests passed with the pinned httpx dependency; this warning is not suppressed.

A genuine integration issue was found and fixed during validation: Vault's unseal response can arrive before the Raft leader becomes active. Bootstrap now waits for a healthy active leader before writing mounts and policies. The subsequent native Vault test passed.

A final startup review also checked two cases with mocked Compose: the development image is built, while a selected release image is preserved on restart. Both passed; this is an orchestration check, not a container execution test. Shell syntax was checked after the change.

## Your acceptance sequence

```
make up
make smoke
make rotate-key
make backup
make restore-test FILE=.backups/ACTUAL_FILE.dump
make drill
make full
make verify-full
make config-check
```

For the optional AWS extension, run `terraform fmt -check`, `terraform init`, `terraform validate` and an account-specific `terraform plan` before considering an apply. For Kubernetes, complete `make k8s-up` and inspect pod health in the explicitly named context.

Do not copy this record into a CV as proof that you personally operated the stack. Complete the exercises, collect your own results and describe this as a personal learning project.

# 15 / Learning plan, interview demo and CV

Use this as a six-week sequence at roughly 8-12 focused hours per week, not a promise of how fast you will learn. Rebuild and explain each layer yourself. Commit your own decisions and incident notes.

| Week | Work | Demonstrable finish |
|---|---|---|
| 1 | Linux processes, permissions, TCP/DNS/TLS; inspect Dockerfile, API, volumes and networks | Explain one request from browser to database; run CRUD and stop/restart without losing data |
| 2 | NGINX, TLS, CRS blocking, rate limits and secrets hygiene | Show a legitimate request succeeding and local attack probes returning 403; explain a false positive |
| 3 | Vault lifecycle, policies, AppRole, Agent and rotation | Show a read-only token unable to write; rotate a key without restarting the API |
| 4 | Prometheus, PromQL, Grafana, Alertmanager, JSON logs and ELK | Trigger a real alert, correlate a request ID and write an incident note |
| 5 | Git branches/PRs, CI gates, image tags/digests, release/rollback, backup/restore | A green pipeline on your repository; a recovered failed release; an isolated restore with measured duration |
| 6 | Terraform state/plan, AWS basics, Ansible idempotence and kind | Explain the cloud plan; complete one optional VM or Kubernetes exercise and record actual results |

Your highest-value additions for internships are Linux/networking fluency, Git/CI/CD, Bash/Python, Terraform, one cloud provider and Kubernetes basics. Do not postpone applying until you know every tool in this repository. Show clear evidence for the parts you have actually practiced.

## Suggested Git issues

1. Explain and diagram the request and secret flows.
2. Rebuild the API image and prove it runs without root.
3. Verify TLS and WAF behavior; record a harmless false-positive investigation.
4. Demonstrate Vault policy denial and API-key rotation.
5. Add one useful dashboard panel and justify the metric.
6. Diagnose a database outage using readiness and logs.
7. Restore a backup and record task count and duration.
8. Run CI on a pull request and explain each failure gate.
9. Deploy a new image and perform a rollback.
10. Complete the kind exercise or optional Terraform/Ansible VM.

For each issue, write acceptance criteria before implementation and attach your own command output or screenshot after it passes. Do not fabricate green badges or timings.

## Ten-minute interview demonstration

| Time | Show | Explain |
|---|---|---|
| 0-1 min | Architecture diagram and `make status` | Ownership boundaries and why profiles exist |
| 1-2 min | Web board and a task | Request path, authentication, persistence |
| 2-3 min | `make smoke` | WAF versus application validation; trusted TLS |
| 3-4 min | `make rotate-key` | Policy, AppRole and Agent render lifecycle |
| 4-6 min | Grafana, then `make drill` | Scrape versus end-to-end probe; alert pending/firing/resolved |
| 6-7 min | Kibana request ID, if full mode is ready | Structured logs and what you deliberately do not log |
| 7-8 min | Isolated restore evidence | Backup success versus recoverability |
| 8-9 min | CI run and tested image tag | Artifact identity and release gates |
| 9-10 min | Your incident note and next improvement | Tradeoffs, limitations and what you learned |

Pre-pull images and practice before the interview. Keep all keys and root material off-screen. If full ELK does not fit your laptop, show real previously captured evidence and say so.

## CV wording - use after you run and understand the relevant features

**English title:** AtlasOps - DevOps / DevSecOps Portfolio Lab

- Built a Docker Compose platform for a Python/PostgreSQL service with NGINX TLS, ModSecurity/OWASP CRS, and Vault AppRole-based secret delivery.
- Automated CI tests, container security scans and versioned image publishing; implemented health-checked releases, rollback and isolated database restore drills.
- Instrumented the service with Prometheus/Grafana and centralized JSON logs with Elasticsearch, Logstash and Kibana; documented incident investigation and recovery.

**French title:** AtlasOps - Projet personnel DevOps / DevSecOps

- Conteneurisation d'une API Python/PostgreSQL avec Docker Compose, reverse proxy NGINX/TLS, WAF ModSecurity/OWASP CRS et gestion des secrets avec Vault/AppRole.
- Automatisation des tests CI, scans de securite et publication d'images versionnees ; deploiement avec controle de sante, retour arriere et tests de restauration.
- Supervision avec Prometheus/Grafana et centralisation des logs JSON dans ELK ; redaction de procedures d'exploitation et de reprise.

Add Terraform/AWS, Ansible or Kubernetes to these bullets only after completing the corresponding exercise. Add a number only if you measured it, such as your actual restore duration or test count. Label this **personal project**, not employment or production experience.

## Questions to prepare

- What is the difference between a process being alive and a service being ready?
- Why is Docker group membership effectively root access?

- Why can changing a secret file fail to rotate an existing PostgreSQL password?
- How does Vault Agent get its first credential, and what happens when that credential expires?
- Does stopping Vault instantly revoke a key already cached by the application?
- Why does a WAF not replace parameterized SQL or authentication?
- Why are route templates better than task IDs as Prometheus labels?
- What does p95 mean, and why is it misleading with almost no traffic?
- How do you prove a backup is recoverable?
- Which tool owns infrastructure, host configuration and application deployment?
- What could make an image rollback fail after a schema change?
- What would you change before exposing this system to the internet?

# 16 / Architecture decisions

## ADR 001: a small application, substantial operations

The task board has CRUD endpoints, a persistent database and a shared API key. Its purpose is to make release, observability and recovery behavior visible. A complex business domain would consume time without strengthening the targeted internship skills. The consequence is that this is not a complete multi-user SaaS application.

## ADR 002: Compose as the reference platform

The learner already understands Docker and NGINX. Compose provides a small reproducible environment where network and lifecycle behavior can be explained. Kubernetes is a separate migration lab so orchestration concepts can be learned without duplicating every operational system at once. Neither is high availability on a single machine.

## ADR 003: two proxy layers

The outer NGINX owns certificates, safe access logging, rate limiting and public route restrictions. The OWASP image owns ModSecurity and the CRS lifecycle. This uses a maintained WAF image and keeps the NGINX configuration familiar. It costs one extra hop and requires recreating the WAF when the API address changes.

## ADR 004: an actual Vault server and Agent

Vault uses persistent integrated storage and explicit initialization/unseal, not development mode. AppRole grants only reads on one KV v2 path; Agent renders a key into a file that the app reloads. The local root recovery material and one recovery share are intentional lab compromises. PostgreSQL passwords remain Compose file secrets: this project does not claim Vault dynamic database credentials.

## ADR 005: file-based log collection

API and NGINX write sanitized JSON into a shared volume. Logstash reads the files, persists offsets and queues events. This avoids giving a collector privileged Docker socket access. It requires rotation management, can duplicate events after failure, and is not a complete fleet-wide logging design.

## ADR 006: controlled release and optional cloud

CI validates and publishes the exact image that passed runtime checks. A host-side command performs the release and rollback. This avoids embedding permanent SSH credentials into CI and makes deployment observable to the operator. It has a brief interruption and is not automatic continuous deployment. Terraform provisions a private demo VM reachable through a restricted SSH rule, with dashboards still on loopback.

# 17 / Repository map and acceptance checklist

## Where to read the code

| Path | What to inspect first |
|---|---|
| app/main.py | Factory, lifespan, key lookup, middleware, error handler and CRUD routes |
| app/static/ | Browser board; memory-only key handling and textContent rendering |
| Dockerfile | Wheel builder, runtime user and readiness health check |
| compose.yaml | Service dependencies, profiles, ports, mounts and networks |
| .env.example | Readable image versions and baseline manifest digests |
| infra/postgres/init.sh | Application role creation on first database initialization |
| infra/nginx/nginx.conf | TLS, limits, proxying, safe access log format and metrics denial |
| infra/vault/ | Persistent server, Agent template and minimal policy |
| infra/prometheus/ | Scrape jobs, rules and the alert timing test |
| infra/grafana/ | Provisioned Prometheus source and six-panel dashboard |
| infra/logstash/ | Persistent queue, file input, filtering and index output |
| scripts/ops.py | Initialization, Vault bootstrap, smoke, rotation and telemetry checks |
| scripts/deploy.sh | Release gate, previous-image tracking and automatic recovery attempt |
| scripts/backup.sh / restore-test.sh | Dump integrity and isolated restore |
| tests/ | API, configuration, PostgreSQL and optional native Vault tests |
| .github/workflows/ci.yml | Test, scan, stack, artifact and publication jobs |
| terraform/aws/ | Optional cloud environment and provider lock |
| ansible/ | Dedicated Ubuntu host preparation |
| k8s/ | Separate application/database migration exercise |
| docs/ | Runbooks, decisions, job evidence, learning plan and honest validation record |
| tools/ | Rebuild this handbook from its Markdown source |

## Before publishing your own repository

Use `git status --short` and inspect exactly what will be committed. The delivered archive excludes generated credentials and runtime data. After you run the project, your working directory will contain private files; never add them with a forced recursive command. Review `.gitignore` and `.dockerignore` yourself.

```
git init
git add .
git status --short
```

```
git commit -m "Add AtlasOps learning platform"
```

Create an empty repository in your own GitHub account, add its URL as `origin`, and push your chosen branch. These steps are for you to perform after reviewing the source. The generated workflow publishes packages only after the required jobs succeed.

## Acceptance checklist

- The host prerequisites and full Compose configuration pass.
- The selected profile starts, and the app can create, update and delete a task.
- A normal restart preserves application data.
- HTTPS validation uses the correct CA; management ports remain on loopback.
- Unauthenticated API access fails and public ingress denies metrics.
- Local WAF probes are blocked while ordinary requests succeed.
- The AppRole can read its one path and cannot write or read unrelated paths.
- A new key becomes usable and the old key stops working without API restart.
- Prometheus scrapes the API and the HTTPS probe succeeds.
- A deliberate API stop produces an actual firing alert and recovery clears it.
- If ELK is enabled, real JSON events reach Elasticsearch and Kibana can query them.
- A new backup restores into a separate database and returns the expected data.
- A release and a rollback are exercised with distinct image identities.
- CI results, image references and measured durations are recorded honestly.
- Optional cloud and Kubernetes claims match exercises you actually completed.

Use `docs/EVIDENCE_TEMPLATE.md` for the results. A screenshot of a dashboard with no real traffic is not evidence of monitoring correctness. A checksum alone is not evidence of restore success. A green parser check is not an executed cloud deployment.

# 18 / Primary technical references

All links below were consulted for implementation behavior or verified as official documentation during the research on 5 November 2026. Product documentation and releases can change; use version-appropriate documentation when upgrading. The Moroccan posting links and their retrieval limitations are recorded in Chapter 2.

- D1 - Docker Engine installation on Ubuntu: supported installation path and Compose plugin.
- D2 - Docker Linux post-installation: daemon access and Docker group privileges.
- D3 - Docker Compose service reference: health conditions, restart behavior and service options.
- D4 - Docker Compose secrets: per-service file mounts and the limits of local secret handling.
- D5 - OWASP CRS Docker project: supported images, WAF configuration and audit options.
- D6 - Vault integrated storage: Raft configuration and persistent storage.
- D7 - Vault Agent AppRole auto-auth: RoleID/SecretID files and authentication.
- D8 - Vault Agent templates: KV rendering and static-secret refresh intervals.
- D9 - Grafana provisioning: data sources and dashboard-as-code.
- D10 - Prometheus alerting rules: alert conditions, pending periods and annotations.
- D11 - Logstash file input: file positions and rotation behavior.
- D12 - Elasticsearch Docker guidance: container memory and host requirements.
- D13 - GitHub publishing Docker images: registry authentication and publication permissions.
- D14 - Terraform AWS EC2 instance reference: VM resource arguments and outputs.
- D15 - Ansible apt module: package-state management.
- D16 - Kubernetes probe configuration: startup, readiness and liveness semantics.
- D17 - Trivy documentation: vulnerability, secret and image scanning.
- D18 - kind quick start: local clusters and loading local images.
- D19 - Vault Raft snapshot restore: isolated snapshot restore procedure.
- D20 - Prometheus rule unit testing: time-series fixtures and alert assertions.

The source repository is the authoritative implementation. This PDF explains it; it does not replace the code, pinned versions, tool diagnostics or your own runtime acceptance record.